

IC-HACT: Frontier-Aware Verification with Soundness-Separated Learning

Research manuscript*

September 18, 2026 | Version 2.0

Abstract

A large coding-agent system needs a compact answer to a precise question: which checks passed on the candidate that is about to be accepted? A tree of verification certificates bounds individual messages, but arranging that tree using only invalidation events is insufficient when checks finish progressively and verification can reject a candidate early. We formulate certificate layout as an optimization over *completion frontiers*, including the full construction cost of each new immutable candidate epoch. A four-obligation separation proves that even identical joint invalidation distributions can require different optimal layouts. An exact ordered-tree compiler follows from classical subtree/forest dynamic programming. We combine it with intervention-conditioned test prioritization while excluding all learned edges from the trusted acceptance predicate: historical evidence changes ordering, never the obligation to establish a complete, current pass. Our implementation includes a strict pytest wrapper, source/checker/environment identities, bounded packets, and epoch-fenced evidence. Evaluation executes 121 source mutations across two upstream Python libraries and the Future Code runtime: 45 training cases, 43 development cases, and 33 fresh, nonoverlapping confirmation cases introduced after a disclosed negative pilot result. On 33 completed confirmation candidates, the pooled frontier tree reduces measured aggregation traffic by 10.33% relative to a compact eight-ary tree. Among 23 rejected candidates, conditional ordering reduces checks to first failure by 47.34% relative to call-profile ordering. Actual timing does not support an equivalent speedup: the paired process-time reduction versus that baseline is 1.58%. Every recorded timing-run verdict agrees with its complete scoped oracle. All aggregation packets fit in 3072 bytes. These are verification and feedback measurements, not evidence of improved LLM reasoning, autonomous repair, or distributed consensus. The result is a concrete, reproducible way to optimize evidence delivery without making acceptance depend on the accuracy of a learned impact graph.

Keywords: agentic software engineering; regression testing; bounded context; completion frontiers; executable evidence; workload-aware trees.

1 Introduction

An agent can finish editing a file long before the system has established that the resulting candidate satisfies its acceptance contract. With many workers, old test reports, partially checked branches, and changing integration states can coexist. A manager that reads every transcript is expensive; a manager that reads only optimistic summaries may be wrong. The relevant systems object is therefore not another managerial conversation, but a compact, current, machine-checked projection of the declared verification state.

Our earlier HACT prototype separated the task graph from an ordered certificate tree. It optimized certificate refresh traffic for jointly invalidated checks. That model omitted an important practical distinction: *becoming stale and becoming checked are different events*. After a candidate changes, a conservative verifier may invalidate the entire test registry. Yet a useful schedule may detect a defect after one check, whereas a poor schedule runs many checks before finding the same defect. Intermediate completions, not just the initial stale set, determine which certificate ancestors must be refreshed.

*Author names and affiliations are to be confirmed before submission. Mathematical development, software, experiments, and writing were AI-assisted. This manuscript is not an accepted conference paper.

This paper studies that gap. IC-HACT uses a source-change descriptor and controlled mutation outcomes to order tests, then compiles the resulting completion-frontier distribution into a bounded certificate tree. Crucially, this information is advisory. The strict implementation requires every approved obligation to pass on a matching candidate/checker/environment identity before it authorizes acceptance. A missing edge in the learned graph can cause wasted work or a poor schedule; it cannot turn an unexecuted obligation into a current pass.

A concrete use case. A coding worker proposes a candidate affecting a graph traversal routine. Historical interventions indicate which checks often fail after modifications to that routine. The harness runs those checks early. If one genuinely fails, the worker receives its current failure evidence and can revise the candidate immediately. If none fail, the harness continues through the entire approved registry. A small certificate tree records progress in bounded messages; a root pass is available only at complete closure. The next revision starts a fresh epoch, so delayed evidence from the previous attempt cannot certify it.

Neither test prioritization nor dependency graphs are new. Version-specific prioritization and greedy fault-exposure ordering are established [1]; regression-test selection has extensive prior art [2, 3]. We do not present these techniques as inventions. The new research object is the distribution of *schedule-induced completion frontiers* under a strict evidence predicate, its separation from invalidation information, and a measured compiler that realizes the corresponding traffic objective. Learning can assist the control plane without being entrusted to define correctness.

Contributions. First, we derive a frontier-sensitive objective and prove an information separation: joint invalidation statistics, even when known exactly, are insufficient for optimal progressive certificate layout. Second, we give an exact specialization of classical alphabetic dynamic programming and a conservative holdout-based switching rule that prices reconstruction and uncertainty. Third, we implement the soundness separation in an executable harness and evaluate real source mutations, actual pytest processes, serialized packets, and adversarial state transitions. The study preserves an unfavorable result: per-source tree specialization overfits the small training set. We select the pooled-frontier variant only as a disclosed development decision, then evaluate it on newly frozen, nonoverlapping intervention sites.

2 Related Work and Attribution

Testing and change impact. Rothermel et al. [1] formulate test prioritization over permutations and investigate fault- and coverage-based orderings. Our empirical fault-set cover per unit cost is an instance of that established family, not a novel ranking algorithm. Ekstazi [2] tracks file dependencies to select tests after changes. NameRTS [3] uses a finer Python name–element graph and prior execution information. Unlike those selection techniques, our strict mode does not omit an approved obligation when issuing a pass. Our file-call baseline is an intentionally simple prioritizer, *not an implementation of NameRTS or Ekstazi*. Consequently, we claim no benchmark superiority over either system.

Incrementality and authenticated layouts. Self-adjusting computation [4] and build-system designs [5] already separate dependency reasoning from recomputation. Generalized alphabetic-tree optimization [6] provides the subtree/forest recurrence used below; its asymptotic complexity is inherited. Workload-adaptive authenticated structures, including the recent Huffman–Merkle Tree [7], make layout and maintenance cost workload-dependent. Our objective instead counts bounded verification-aggregation packets on completion waves while holding leaf order and the acceptance registry fixed. The local certificates are not zero-knowledge proofs, signatures, or adversarial remote attestations.

Agent runtime assurance. Proof-Carrying Agent Actions [8] addresses action-bound governance across heterogeneous runtimes. VP-Control [9] studies verification portfolios, shared-source failures, and commit-

Table 1: Novelty boundaries. An inherited mechanism is not relabeled as an invention.

Aspect	Established foundation	Research contribution investigated here
Task coordination	Hierarchy, narrow contexts, durable state	Separate task ownership from schedule-dependent evidence layout
Change intelligence	Mutation testing, dependencies, prioritization	Advisory intervention graph coupled to an independently enforced complete registry
Layout algorithm	Alphabetic subtree/forest DP	Completion-frontier weights, initial-epoch price, and an invalidation-information separation
Runtime correctness	Hash binding, fencing, trusted checkers	Executable realization and tests of invariance under learned-graph and layout errors
Evaluation	Fault injection and replay	Upstream executable cases, direct timing, actual packets, and fresh confirmation after a negative pilot

time enforcement. These are relevant predecessors to any claim about trustworthy agent actions. Our epoch guard borrows established fencing and complete-coverage principles. It supports a narrower claim: advisory scheduling and tree changes cannot change the local acceptance predicate. It does not make an arbitrary external deployment transaction atomic.

3 Model: Separate Advice from Acceptance

3.1 Obligations and immutable candidate epochs

Let $\mathcal{R} = (g_0, \dots, g_{n-1})$ be a nonempty, ordered, approved registry of executable obligations. An obligation is not an agent: one worker may discharge many checks, and an integration check may concern several workers. A candidate epoch has identity

$$e = (\text{epoch}, \text{snapshot}, \text{checker}, \text{environment}, \text{registry}). \quad (1)$$

The snapshot fingerprint covers the frozen source manifest, including acceptance inputs. The checker and environment identities are recorded separately. In the local wrapper the operator first approves a contract on a trusted baseline; protected test and configuration hashes remain outside candidate write scopes.

For each g_i , the trusted runner can submit a current outcome in $\{\text{PASS}, \text{FAIL}, \text{UNKNOWN}\}$. Missing, timed-out, uncollected, incompatible, or untrusted evidence is unknown. A candidate is accepted only when the trusted predicate

$$A(e) = \mathbf{1}\{\text{every } g_i \in \mathcal{R} \text{ has current, matching PASS evidence for } e\} \quad (2)$$

holds. A fail permits early rejection, not an assertion that the remaining checks have been executed. A pass is relative to this registry, not a proof that arbitrary program behavior is correct.

3.2 The advisory intervention graph

A baseline profile records calls from tests to source files. Separately, each controlled intervention changes one AST site in a private source copy and executes the complete scoped suite. The observed failing test set is a hyperedge attached to that source-change descriptor. Since the unmodified baseline passes, the experiment establishes an effect of that particular intervention under that particular fixture. It does not establish a complete structural causal model, infer all future dependencies, or justify counterfactual conclusions about unobserved multi-file changes.

The advisory graph maps a change descriptor to an ordering π_e , an exact permutation of $\mathcal{R}$. It can be learned incorrectly, lose edges, or encounter a new source. None of these events changes Equation (2).

Cold-start fallback is a deterministic call-profile/cost ordering, with every remaining obligation retained in the tail. User-authored prose cannot submit trusted outcomes or rewrite the registry.

3.3 Bounded certificate interface

A certificate node covers a contiguous registry interval $I_v = [i, j]$. Its children must partition that interval without gaps or overlaps. A child card carries interval bounds, counts of pass/fail/unknown obligations, registry identity, input binding, and an evidence commitment. The kernel computes these values; they are not generated by an LLM. Individual logs and detailed failure traces remain in durable storage and are retrieved by identifier.

An internal node with d children emits a fixed-width packet of

$$c(d) = H + Sd, \quad 2 \leq d \leq b = \left\lfloor \frac{C - H}{S} \right\rfloor. \quad (3)$$

The evaluated format uses $H = 512$, $S = 320$, and $C = 3072$ bytes, so $b = 8$. Overlong mandatory fields cause rejection rather than truncation. Bytes are not model tokens. The cap bounds one verification message; it does not bound every agent prompt, total project memory, total communication, or retrieval latency.

Proposition 1 (Soundness separation). *Assume (i) a complete approved registry, (ii) a trusted checker executing the identified immutable inputs, (iii) collision-resistant identity binding, and (iv) an atomic local epoch/ticket check. For any advisory graph, any complete test ordering, and any structurally valid certificate layout, authorization of a root pass implies Equation (2). Arbitrary changes to the advisory graph cannot create a false pass relative to the registry.*

Proof. A leaf can contribute a pass only through current trusted evidence matching its obligation and epoch. Every internal node sums the three counts of children whose intervals form an exact partition; induction therefore preserves both coverage and the meaning of each count. The root is authorized only for counts $(n, 0, 0)$ on the current registry and epoch. Changing the graph cannot manufacture a leaf outcome. Changing the layout must re-establish the same partition and commitments. The final local epoch check rejects a ticket invalidated by a concurrent change. $\square$

The assumptions matter. An incomplete test suite, a compromised checker, nondeterministic behavior outside the recorded environment, or an unprotected external deployment can still invalidate an application-level correctness claim. Hashing an incorrect assertion does not make it true.

4 Completion-Frontier Optimization

4.1 Why an invalidation set is insufficient

A strict new epoch invalidates all obligations. If an optimizer sees only that set, every candidate looks identical. In contrast, after test prioritization a failing candidate may need only a short executed prefix. During a successful candidate the same interval may become active repeatedly as its descendants finish. The useful workload unit is thus an *ordered sequence of completion sets*, not a single initial invalidation set.

Definition 2 (Completion frontier). For epoch e , let $W_{e,1}, \dots, W_{e,K_e}$ be the nonempty sets of obligations whose final outcomes are submitted between consecutive certificate refreshes. A schedule and an outcome vector induce this sequence. The evaluated sequential harness stops at its first failed test and groups the executed prefix into waves of at most eight completed tests; it flushes the final partial wave. Other batching policies induce different sequences.

Table 2: The separation in units of c . Every row sees the same initial invalidation set. The table includes initialization, not only incremental updates.

Tree family	Leaf depths	Workload A	Workload B	Maximum regret
Root-0 comb	(1, 2, 3, 3) or (1, 3, 3, 2)	4	12	1
Balanced	(2, 2, 2, 2)	5	11	1
Other two ordered shapes	(2, 3, 3, 1) or (3, 3, 2, 1)	5 or 6	12	1 or 2

A fresh strict epoch incurs one complete certificate construction, even when it fails immediately. With a tree T , its realized aggregation traffic is

$$L_e(T) = \sum_{v \in \text{int}(T)} c(d_v) + \sum_{k=1}^{K_e} \sum_{v \in \text{int}(T)} \mathbf{1}\{W_{e,k} \cap I_v \neq \emptyset\} c(d_v). \quad (4)$$

This charges internal aggregation packets, including initialization. It excludes raw test logs, source snapshots, transport framing, and model requests. We also measure actual serialized packet lengths to check the formula.

Taking expectation gives the sufficient interval statistic

$$r(i, j) = 1 + \mathbb{E} \left[\sum_{k=1}^{K_e} \mathbf{1}\{W_{e,k} \cap [i, j] \neq \emptyset\} \right], \quad (5)$$

$$J(T) = \sum_{v \in \text{int}(T)} r(I_v) c(d_v). \quad (6)$$

Unlike an activation probability, $r(i, j)$ can exceed one. It includes repeated progress updates and the unconditional initialization cost. Replacing it with $\mathbb{P}(U_e \cap [i, j] \neq \emptyset)$ loses information even when the entire distribution of U_e is known.

4.2 A strict information separation

Theorem 3 (Joint invalidation does not determine optimal progressive layout). *Consider four ordered obligations, binary certificate trees, and internal packet price $c > 0$. There exist two workloads with identical joint invalidation distributions for which the minimum frontier costs are $4c$ and $11c$, respectively, and the optimal tree families are disjoint. Any randomized selector given only the invalidation distribution has worst-case expected additive regret at least $c/2$. For deterministic selectors the bound is c .*

Proof. Every epoch invalidates $\{0, 1, 2, 3\}$ and uses order $(0, 1, 2, 3)$ with one completion per wave. In workload A, obligation 0 always fails, so only its wave is executed. A binary tree has three internal nodes; initialization costs $3c$. Its total cost is $(3 + \text{depth}(0))c$, minimized at $4c$ by making leaf 0 a child of the root. In workload B, obligation 3 always fails and all four checks execute. Its cost is $(3 + \sum_i \text{depth}(i))c$, minimized at $11c$ by the balanced tree. A root-0 comb has leaf-depth sum nine and costs $12c$ in B; the balanced tree costs $5c$ in A. The remaining ordered binary trees are weakly dominated by these choices. If the selector chooses a root-0 comb with probability p , the two regrets are at least $(1 - p)c$ and pc . Their maximum is at least $c/2$, with equality at $p = 1/2$. A deterministic choice incurs at least c in one workload. $\square$

The result is deliberately narrow: it concerns the information supplied to a layout selector, not a universal impossibility for adaptive algorithms. A selector that observes completion outcomes has more information and is not covered by the lower bound.

4.3 Exact ordered-tree compilation

For a nonnegative interval matrix r , let $D(i, j)$ be the minimum cost of a subtree covering $[i, j]$. Leaves have $D(i, i) = 0$. Let $F_k(i, j)$ be the minimum cost of an ordered forest of k nonempty trees partitioning $[i, j]$. Using infeasibility value $+\infty$,

$$F_1(i, j) = D(i, j), \quad (7)$$

$$F_k(i, j) = \min_{m=i+k-2}^{j-1} \{F_{k-1}(i, m) + D(m+1, j)\}, \quad k \geq 2, \quad (8)$$

$$D(i, j) = \min_{2 \leq k \leq \min(b, j-i+1)} \{r(i, j)c(k) + F_k(i, j)\}. \quad (9)$$

Intervals are processed by increasing length. The apparently circular $F_1 = D$ term is only used on shorter intervals when building a forest with at least two components.

Proposition 4 (Exact compilation). *Equation (9) returns an ordered tree minimizing Equation (6) under the packet cap in Equation (3). It requires $O(bn^3)$ time and $O(bn^2)$ storage in the direct implementation. Empirical completion waves can replace the expectation without altering exactness for the empirical objective.*

Proof. Any feasible internal root has k ordered children forming a contiguous partition. Its contribution depends only on the parent interval and k ; remaining contributions are independent subtree costs. The forest recurrence enumerates every such partition. Induction on interval length therefore proves optimal substructure and exhaustive coverage. There are $O(bn^2)$ forest states, each scanning at most n splits. $\square$

This proof instantiates the classical alphabetic-tree template [6]. The research distinction lies in the workload statistic and the resulting separation, not in claiming a new generic DP. For numerical compatibility with the earlier optimizer, the code divides the empirical matrix by its maximum and rescales the optimum; a positive common scaling cannot change the minimizing tree. Leaf order is held fixed. We neither solve unrestricted graph partitioning nor jointly optimize all test schedules and tree shapes.

5 Learning and Conservative Adaptation

5.1 Intervention-conditioned priority

The training dataset contains source descriptors x_m and failing obligation sets B_m from complete executions. For a candidate source x , retain matching nonempty failure sets. At each greedy step, select the remaining check that covers the largest number of still-uncovered training failure sets per baseline measured duration. Remove the covered sets and continue. Append all other obligations in a deterministic call-profile/cost order. If there are no matching failure sets, use the complete fallback order.

The global-history ablation performs the same greedy operation without conditioning on x . The shortest-first ablation ignores both failure sets and source information. Baseline execution durations are floored at 10^{-4} seconds to avoid division by zero; nonfinite or negative inputs are rejected. Stable registry indices resolve ties. These choices are recorded before evaluating unseen mutations.

This procedure does not produce a calibrated failure probability. Three training mutations in one source file can easily be unrepresentative. In particular, treating a missing edge as evidence that a check is irrelevant would be unsound. IC-HACT uses the graph only to decide *what to check next*, not *what need never be checked*.

5.2 Pooled and conditional layouts

Once the ranker is fixed, replay its ordering on training outcome vectors to obtain training completion waves. A pooled-frontier tree fits Equation (5) across all training cases in a project. A more aggressive conditional

variant fits a separate tree per source descriptor. Both use exactly the same acceptance semantics. The latter can reduce training cost but has fewer observations and a larger model family.

Our pilot showed that conditional layouts could be substantially worse out of sample. Accordingly, the revised default is *conditional priority plus a pooled-frontier layout*, with conditional layouts retained as an ablation. This decision used the development results. To avoid calling a post-selection result untouched test evidence, we subsequently froze 33 new, nonoverlapping mutation sites and left all models and serialized trees fixed for confirmation.

5.3 A sufficient holdout switching gate

Changing layouts has a price. Beyond initial construction already counted per epoch, a live system may need an additional reconstruction, announcement, and old-ticket revocation. Let S_{switch} be that measured cost in the same units as L . Suppose K candidate plans are fixed before an independent validation sample of size m . Each incumbent and candidate episode cost lies in $[0, B]$. Let $\bar{\Delta}_j$ be paired mean savings for candidate j .

Proposition 5 (Conservative promotion). *Under independent, identically distributed validation episodes and a stationary deployment distribution, with probability at least $1 - \delta$, every fixed candidate has true expected savings at least*

$$\underline{\Delta}_j = \bar{\Delta}_j - B\sqrt{\frac{2\log(K/\delta)}{m}}. \quad (10)$$

For an intended h -epoch horizon, $h\underline{\Delta}_j > S_{\text{switch}}$ is a sufficient condition for positive expected net savings on the event above.

Proof. The paired difference lies in $[-B, B]$, an interval of width $2B$. Applying the one-sided Hoeffding bound [10] and a union bound across K fixed candidates gives Equation (10). Multiplying expected per-epoch savings by h and subtracting the switching price gives the final condition. $\square$

This is a sufficient condition, not a guarantee that a small pilot permits adaptation. With few validation samples and a broad worst-case bound it can reject every switch. It is also not a drift-robust bound. If the workload shifts, the system should retain correctness through the strict gate, fall back conservatively, and collect a new held-out performance sample. Byte savings cannot be silently exchanged for latency savings; pricing time and bytes jointly requires an explicitly declared conversion.

6 Executable Realization

The research artifact contains the original HACT kernel and tests, a frontier compiler, an intervention ranker, a pytest evidence recorder, an epoch guard, an executable strict wrapper, and independent analysis scripts. It vendors the exact source snapshots used in the experiments. No hosted LLM is required to reproduce these measurements.

Trusted versus advisory components. The ranker and fitted tree are advisory. A structurally invalid tree is rejected; a valid but poor tree can only increase cost. The recorder supplies setup, call, and teardown outcomes. A pass requires all three phases to pass; missing or skipped evidence remains unknown. The registry is collected and checked as an exact identity list. The protocol records source, checker, environment, and result fingerprints. These are local integrity bindings, not remote signatures.

Concurrency and contradictory evidence. Each submitted result carries an epoch token. A new candidate fences old tokens, including ABA transitions in which source bytes return to an earlier value. Exact delivery retries are idempotent. Conflicting results for the same obligation in one epoch revoke

Algorithm 1 IC-HACT strict candidate verification

Require: Approved registry and acceptance inputs; immutable candidate identity e ; advisory model; valid bounded tree T

- 1: Invalidate prior-epoch evidence; construct the complete unknown certificate tree
- 2: Obtain an exact registry permutation π ; reject any omission or duplicate
- 3: **for** each check in π **do**
- 4: Execute the approved checker on the frozen candidate
- 5: Bind the final result to the obligation, epoch, checker, environment, and trace
- 6: Reject stale results; revoke authorization on conflicting same-epoch evidence
- 7: **if** a completion wave closes or a failed check ends the attempt **then**
- 8: Refresh only certificate ancestors intersecting the wave
- 9: **end if**
- 10: **if** current trusted result is FAIL **then**
- 11: Return failure evidence; do not report remaining checks as complete
- 12: **end if**
- 13: **end for**
- 14: Require complete current pass coverage and an atomic local epoch check
- 15: Return a candidate-specific local authorization, not an external deployment action

authorization and require a fresh epoch; a late pass cannot erase a recorded fail. A local lock protects evidence submission, root refresh, and ticket checks. The implementation does not lock an arbitrary external repository or provide a distributed transaction.

Usable strict pytest wrapper. The operator initializes an approved contract on a trusted baseline, stores it outside candidate write scopes, and verifies subsequent copies against it. The wrapper checks protected test/configuration hashes, rejects symbolic links, records a full source manifest, executes in a private copy, detects source changes during the run, and emits a fixed report plus actual certificate packets. It never deletes user source or performs a deployment. Private copying is not an OS sandbox: hostile candidate code requires a separately configured container or VM, restricted credentials, and a trusted checker boundary.

Scope of the optimization. The existing Future Code hierarchy is not replaced by this certificate tree. Tasks retain their own ownership and integration dependencies. The wrapper can serve as a candidate-stage verification component, but this artifact does not claim a completed production integration or a 512-live-agent deployment. In particular, the test registry size must not be relabeled as an agent count.

7 Evaluation Design

We ask four questions. **RQ1:** Can a source-conditioned schedule return genuine failure feedback with fewer executed tests and lower measured process time? **RQ2:** Does a frontier-aware tree improve measured certificate traffic after charging initialization? **RQ3:** Do missing, stale, conflicting, or adversarially incomplete records ever obtain a local pass? **RQ4:** Which costs and negative results limit practical value?

7.1 Executable subjects and scope

We use exact local source copies of NetworkX [11], Toolz [12], and Future Code Control v1.2.0. The first two are upstream libraries with their unmodified existing tests. The third is the uploaded agent-runtime prototype, not an independent industrial deployment. Source copies and licenses are included in the artifact. A public benchmark download was unavailable because container DNS resolution failed; we did not substitute invented external-benchmark results.

Table 3: Executable subjects. Mutation cells report rejected / surviving / unresolved cases, not autonomous repairs. A surviving mutant is not a proof of semantic equivalence.

Subject	Tests	Training	Development	Confirmation
NetworkX	170	11/5/0	9/5/1	5/4/0
Toolz	148	9/2/0	8/2/0	6/0/0
Future Code	138	8/10/0	10/8/0	12/6/0

The frozen scopes contain 170 NetworkX tests for components, traversal, and unweighted/generic shortest paths; 148 Toolz tests for iterable, functional, dictionary, recipe, curried, and utility behavior; and 138 Future Code core tests for contracts, state, coordination, and hierarchy. All scoped baselines passed. Separately, the original Future Code full suite passed all 263 tests. The mutation study does not claim to rerun that full 263-test scope on every changed candidate. An initially considered SymPy scope was excluded before mutation outcomes because its upstream test configuration required an unavailable dependency; the replacement with Toolz is recorded in the contract.

7.2 Predefined interventions and disclosed development decisions

For each of 17 predefined source files, we select up to six nonoverlapping AST sites by a fixed seeded hash. Operators alter relational boundaries, arithmetic operators, Boolean constants, or negation. Only the selected expression span changes; the surrounding file is not reformatted. Selected sites alternate between training and development. This yields 45 training and 43 development cases. We retain surviving mutants, timeouts, and collection failures rather than replacing them with easier-to-detect faults. A surviving mutant means only that the scoped tests did not reject it.

After the development outcomes revealed conditional-tree overfitting, we fixed pooled-frontier layout as the revised default and froze 33 additional intervention sites with a new seed. These sites do not overlap any training or development mutation span, including nested expressions. They span 11 source files; some files had too few unused sites. The original 45 training cases, ranker, and serialized plans remain fixed. Confirmation outcomes do not refit them. Model-plan hashes and the freeze timestamp precede all confirmation executions. This is a local registered experimental contract, not a claim of third-party preregistration.

Every full-oracle execution runs in a fresh private source copy. We check the intended mutant hash and store pytest phase records. A completed oracle must have the exact registry and a final pass/fail outcome for every obligation. Incomplete or timed-out attempts remain unknown and are excluded only from metrics requiring a complete finite oracle; their counts remain visible. Full-oracle runs use four worker processes for throughput. Their durations are not used as controlled latency measurements because of resource contention.

7.3 Baselines and measurements

The prioritization baselines are default collection order, shortest-first, file-call coverage first, global empirical fault-set cover, and source-conditioned fault-set cover. All are full permutations. We replay each ordering against the complete outcome vector to count executed tests to first failure; replayed phase time is explicitly not wall-clock latency.

For actual timing, the development cohort consists of the first eight predefined development IDs per project. After retaining one incomplete oracle as unknown, 23 eligible snapshots are executed with four orderings and three repetitions, for 276 fresh subprocess runs. The confirmation timing cohort is the first four confirmation IDs per project, with default, call-profile, and source-conditioned orderings and three repetitions. The exact eligible count and observed agreement are given below. Within each repetition, candidate blocks and ordering labels are deterministically randomized. Repetitions are not treated as

Table 4: Mean executed tests to first failure, among oracle-rejected candidates only. Counts are schedule replay over complete real test outcomes. Successful and unresolved candidates are not in this denominator.

Subject	Split	Failures	Default	Calls	Global	Cond.
NetworkX	Dev.	9	79.4	5.8	40.8	6.0
Toolz	Dev.	8	67.9	25.9	71.2	21.5
Future Code	Dev.	10	67.4	26.6	14.0	5.1
NetworkX	Confirm.	5	79.2	6.0	9.2	1.6
Toolz	Confirm.	6	47.3	35.5	57.3	20.0
Future Code	Confirm.	12	79.5	29.9	34.2	15.8

Table 5: Measured process seconds: means of per-candidate medians from three repetitions. Each cohort includes both rejected and surviving candidates. Includes startup/collection/evidence writing; excludes snapshot copying, offline fitting and certificate replay.

Subject	Split	Cases	Default	Calls	Global	Cond.
NetworkX	Dev.	7	2.832	2.805	2.872	2.841
Toolz	Dev.	8	1.410	1.412	1.449	1.371
Future Code	Dev.	8	2.073	1.835	1.819	1.870
NetworkX	Confirm.	4	2.851	2.912	–	2.813
Toolz	Confirm.	4	1.371	1.408	–	1.402
Future Code	Confirm.	4	1.808	1.716	–	1.726

independent source mutations.

Timing includes process startup, imports, pytest collection, test execution, evidence writing, and order planning. It excludes source copying, offline tree fitting, and certificate replay. A separate strict-wrapper smoke test includes copying, source hashing, and certificate generation. We report per-candidate medians of three repetitions, then aggregate those paired values. Variance from startup and scheduling is real and must not be hidden behind a test-count ratio.

Tree baselines use the same conditional schedule and outcome vectors: (a) a compact bottom-up eight-ary fixed tree; (b) the old invalidation-only exact optimizer, which sees every interval active under strict whole-snapshot invalidation; (c) a pooled-frontier tree; and (d) per-source frontier trees. All trees preserve the same registry order and 3072-byte cap. The invalidation-only optimizer is an objective ablation, not the strongest practical baseline; the compact fixed tree is essential. We run the trusted kernel for each evaluated tree and sum actual serialized packet lengths, including the first full construction. This is a measured protocol replay over real outcomes, not live network transmission.

Uncertainty and estimands. For paired comparisons, the primary reduction is $100(1 - \sum_i y_i / \sum_i x_i)$, where x_i and y_i are baseline and candidate measurements on the same mutation. Confidence intervals use a seeded paired cluster bootstrap, resampling source-file clusters within each project; timing first reduces repeated runs to per-mutation medians. These intervals are conditional on three chosen projects and the intervention design. They do not establish generalization to a population of repositories. We also report project-level values, absolute seconds/bytes, and unfavorable comparisons.

8 Results

8.1 Failure feedback and actual latency

On the fresh confirmation failures, the conditional policy executes a mean of 13.78 checks before rejection, versus 26.17 for the call-profile baseline: 47.34% fewer, with a source-cluster 95% interval [19.26, 77.46]. This is evidence of earlier failure in the test sequence, not automatically faster feedback in seconds. Across

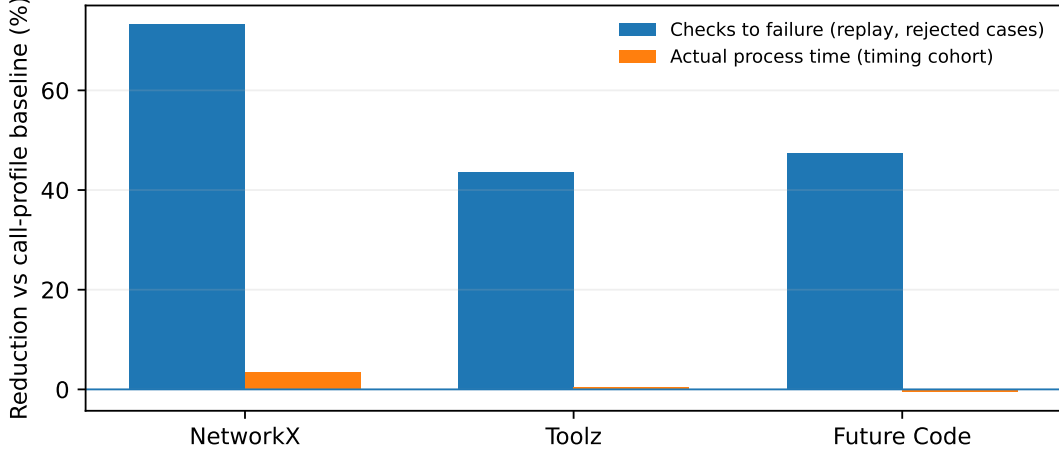


Figure 1: Sequence efficiency is not wall-clock acceleration. Confirmation check-count reductions use all oracle-rejected cases; timing reductions use the separately frozen timing cohort. The two denominators are intentionally labeled and must not be equated.

Table 6: Mean actual aggregation traffic in KiB per completed candidate, including full initial construction. The same conditional schedule is used for every tree; logs, raw evidence and transport framing are excluded.

Subject	Split	Cases	Fixed-8	Inval.	Pooled	Per-source
NetworkX	Dev.	14	288.7	606.2	267.7	414.6
Toolz	Dev.	10	198.9	357.9	183.9	243.6
Future Code	Dev.	18	248.9	480.8	213.9	207.8
NetworkX	Confirm.	9	327.9	709.1	302.2	362.2
Toolz	Confirm.	6	120.4	170.5	113.7	132.7
Future Code	Confirm.	18	225.9	444.7	196.6	191.4

the confirmation timing cohort, mean per-candidate median process times are 2.010 seconds (default), 2.012 seconds (call-profile), and 1.980 seconds (conditional). The paired reduction versus call-profile is 1.58% with interval [0.62, 2.01], compared with 1.48% versus default with interval [-1.06, 5.70]. The development cohort also shows no consistent advantage over call-profile (-0.45%; interval [-1.31, 0.36]). Startup and collection can dominate these short scopes. We therefore do not claim a large or consistently reproduced wall-clock improvement over the stronger ordering baseline.

8.2 Certificate traffic and the negative specialization result

The fresh confirmation result is a reduction from 234.5 to 210.3 KiB per candidate, or 10.33% with source-cluster interval [8.86, 11.66]. The development reduction is 10.22%. Per-source trees are not the selected default: they cost 26.93% more than pooled frontiers on development. On confirmation, pooled frontiers use 7.48% fewer bytes than per-source layouts. Table 6 retains project-level counterexamples, including the Future Code cases where per-source layout is smaller. Initialization is charged on every candidate and no favorable steady-state-only accounting is used.

The invalidation-only objective can produce a deep tree with low one-time construction cost but expensive repeated progress updates. This is not an optimizer bug: it is the wrong objective for that trace. Conversely, a compact fixed hierarchy is a strong baseline, and data-dependent layout is not uniformly superior on every candidate. Lower aggregate bytes do not imply faster execution or higher repair quality.

8.3 Soundness and integrity checks

All 384 actual timing executions agree with their complete scoped oracle verdicts; none of these timing runs timed out. All 300 certificate-layout decisions on 75 evaluable development/confirmation candidates agree with their source oracle. The sole incomplete development oracle remains unresolved; it is not converted to pass. In the controlled edge-pruning replay, the unsafe reuse comparator would accept 8 of 23 confirmation candidates rejected by the full oracle. The unpruned call-profile reuse comparator has no observed false pass in this cohort; the pruning result is a deliberate stress test, not a claim that its natural graph failed. Strict complete-registry authorization does not rely on either graph. The protocol tests exercise complete coverage, missing leaves, stale bindings, same-content and ABA epoch changes, contradictory same-epoch evidence, idempotent retries, invalid layouts, tampered displays, and local concurrent submissions. Exhaustive enumeration of small binary trees checks the compiler independently of the DP recurrence. A separate auditor reads stored trees and outcomes and recomputes packet costs without importing the optimizer or the producer’s pricing helpers. It also checks site disjointness, source hashes, registry equality, timing-record identities, and observed verdict agreement.

We additionally replay a deliberately unsafe comparator that assumes a previous pass can be reused for every test outside an observed call graph. Its edge-pruning ablation removes three quarters of file–test edges by an outcome-independent hash. Any false pass in that ablation demonstrates the danger of allowing incomplete learned relationships to authorize acceptance. It is not a measurement of NameRTS, and it does not estimate an attack rate in production. The strict protocol never converts omitted checks into pass evidence.

8.4 Overhead, batching, and practical boundaries

The pooled compiler takes 0.177–0.302 seconds per project in the width-eight sensitivity run. A synthetic complete-pass kernel-only run with the same registries and five repetitions has a median of 14.2–19.0 ms; this measures the Python protocol, not project tests. The real-project wrapper smoke adds 0.301–0.517 seconds beyond its pytest process for copying, identities, contract checks and certificates. These integrity costs are material relative to the small observed ordering-time differences. We do not claim that the wrapper makes bare pytest faster end to end. The empirical priority needs historic complete outcomes; producing these through deliberate fault injection costs real testing time. A production deployment should normally reuse approved development/repair traces or justify a bounded offline intervention budget, rather than continuously mutating a production checkout. Tree fitting is offline and should be amortized. If startup dominates the suite or few candidates fail, the benefit of reordering can be small. Successful strict candidates still execute the full registry.

9 Practical Use and Limitations

Where the method has a concrete use. IC-HACT is a candidate-verification component for a coding or scientific-programming harness. It is useful when workers repeatedly propose changes, the system can execute a meaningful acceptance suite, and users need small, accurate progress reports. Its immediate value is earlier genuine failure evidence, compact verification-state delivery, and explicit rejection of stale or incomplete pass claims. It is not a replacement for a capable coding model, a good task decomposition, or a carefully designed acceptance contract.

A practical integration keeps source-edit ownership in the existing task DAG. Each candidate enters a read-only or privately copied verification stage. The advisory model proposes an order; the trusted wrapper validates that the order covers the approved registry exactly. Results feed both the certificate tree and a durable evidence store. A failure returns a short structured finding to the relevant worker. A complete pass produces an identity-bound local ticket. An outer integration service must then atomically ensure that the checked candidate is the candidate being merged; this final external transaction is not implemented by our Python lock.

Context discipline. The certificate tree eliminates the need for a model to reconstruct verification state from full conversations. A root can expose counts, failures, unknown obligations, and evidence identifiers without reading every descendant’s logs. However, difficult design questions may still require broad reasoning. The 3072-byte bound applies to a verification packet, not to all reasoning contexts. Tree depth, number of refreshes, raw log size, and operator retrieval remain separate costs. A deep learned tree can worsen progress latency despite respecting the local packet cap.

No causal overclaim. A mutation is an intervention, but an intervention dataset is not automatically a complete causal model. Our graph is an empirical, source-conditioned failure relation under frozen fixtures. It can miss indirect dependencies, new control paths, dynamic imports, environmental effects, and interactions among simultaneous edits. The method’s safety argument deliberately does not require that graph to be complete. Establishing broader causal invariance would require richer interventions and identifiable assumptions, not a change in terminology.

Threats to empirical validity. The study contains two upstream libraries and one related prototype, selected test scopes, synthetic source mutations rather than a representative history of human or LLM repairs, few source-file clusters, and one host. Mutation operators can create unrealistic faults or equivalent changes. Several training files provide very little failure evidence. The confirmation set tests new sites in known projects and mostly known files; it is not an unseen-project transfer study. Interpreting bootstrap intervals outside those boundaries would be unwarranted.

Actual ordering can interact with shared fixtures or order-dependent tests, which is why we compare direct timing-run verdicts with their full-oracle verdicts rather than assuming equivalence. Timeout cases remain unresolved. No experiment evaluates hosted LLM calls, total token consumption, autonomous patch quality, sustained thousands-agent operation, cross-machine scheduling, or high-availability failover. The protocol result is conditional on a trusted checker and accurately identified execution inputs; local source copying alone is not protection against malicious code or external data changes.

What a larger conference evaluation still needs. The manuscript supplies a precise research object, proofs, executable controls, negative results, and a reproducible artifact. It does not itself establish acceptance at a leading venue. A stronger external-validity claim requires independent repositories with real revision histories, standardized agent-generated patch tasks, comparison with fully implemented modern test-selection/prioritization systems, prospective workload drift, and end-to-end integrations that account for preparation, training, verification, and deployment costs. These are explicitly outstanding evaluations, not results deferred to an unavailable appendix.

10 Conclusion

A certificate layout optimized for what becomes stale can be wrong for how evidence becomes available. IC-HACT makes that distinction explicit: the completion-frontier distribution, including first-failure stopping and initial construction, is the relevant object for progressive verification traffic. A small counterexample proves the information gap; an exact inherited-DP compiler realizes the new objective. Controlled source interventions can help prioritize feedback, but learned structure is kept outside the acceptance predicate. The executable study demonstrates both useful effects and limits, including conditional-layout overfitting and startup-dominated latency. The practical design principle is to let learning optimize the route to evidence, while a complete, current, independently enforced contract determines whether the system may claim success.

References

- [1] Gregg Rothermel, Roland H. Untch, Chengyun Chu, and Mary Jean Harrold. Prioritizing test cases for regression testing. *IEEE Transactions on Software Engineering*, 27(10):929–948, 2001. <https://doi.org/10.1109/32.962562>.
- [2] Milos Gligoric, Lamyaa Eloussi, and Darko Marinov. Practical regression test selection with dynamic file dependencies. *Proceedings of ISSTA*, 2015. <https://users.ece.utexas.edu/~gligoric/papers/GligoricETAL15Ekstazi.pdf>.
- [3] You Wang, Michael Pradel, and Zhongxin Liu. Names Are All You Need: Effective and Safe Regression Test Selection for Python. arXiv:2605.25356v1, May 2026. <https://arxiv.org/abs/2605.25356v1>.
- [4] Umut Acar, Guy Blelloch, Matthias Blume, Robert Harper, and Kanat Tangwongsan. A Library for Self-Adjusting Computation. *Electronic Notes in Theoretical Computer Science*, 148:127–154, 2006. <https://doi.org/10.1016/j.entcs.2005.11.043>.
- [5] Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. Build Systems à la Carte. *Proceedings of the ACM on Programming Languages*, 2(ICFP), Article 79, 2018. <https://doi.org/10.1145/3236774>.
- [6] Hirendu Vaishnav and Massoud Pedram. Alphabetic Trees: Enumeration and Optimization with Applications to VLSI Design. Author-hosted manuscript, Sections 4.1–4.4; date not established from the retrieved manuscript. <https://www.mpedram.com/Papers/alp-trees-journal.pdf>.
- [7] Ziheng Shangguan, Aviv Yaish, and Dahlia Malkhi. Authenticated Data Structures for Dynamic Workloads. arXiv:2608.25206v1, August 2026. <https://arxiv.org/abs/2608.25206v1>.
- [8] Zexun Wang. Proof-Carrying Agent Actions: Model-Agnostic Runtime Governance for Heterogeneous Agent Systems. arXiv:2606.04104v1, June 2026. <https://arxiv.org/abs/2606.04104v1>.
- [9] Zihao Zheng, Baichuan Li, Junyi Yao, and Jiayu Long. Engineering Reliable Commit Gates for Agentic AI: Cost-Aware Verification Portfolios under Common-Mode Data Failures. arXiv:2609.10969v1, September 2026. <https://arxiv.org/abs/2609.10969v1>.
- [10] Wassily Hoeffding. Probability Inequalities for Sums of Bounded Random Variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963. <https://doi.org/10.1080/01621459.1963.10500830>.
- [11] NetworkX developers. NetworkX 3.6.1 release documentation and distributed source tests. https://networkx.org/documentation/stable/release/release_3.6.1.html.
- [12] PyToolz contributors. Toolz API documentation and distributed source tests. <https://toolz.readthedocs.io/en/stable/api.html>.

A Proof and Cost-Model Details

A.1 Why the kernel realizes the frontier price

Fix a valid tree and an epoch. On the first refresh, every internal node is uncached and emits exactly one packet of size $H + Sd$. Thereafter, submitting a final outcome marks exactly that leaf and its ancestors as potentially changed. At a refresh, a node’s descendant-generation stamp differs from its cache stamp if and only if at least one submitted obligation lies in its interval. A changed node emits one packet, regardless of

how many changed descendants it has in that wave. Unchanged intervals reuse their current cached capsule. Summing these contributions yields Equation (4).

The implementation additionally validates the entire tree and computes cache keys; these operations have CPU costs that are not equal to packet bytes. The traffic theorem does not assert logarithmic total runtime. It also assumes the stated fixed-width serialization. A different compression or transport format requires re-estimating $c(d)$; reducing padding can change the optimum and the percentage benefit.

A.2 Finite bounds and the role of batching

Without unary internal nodes, an n -leaf tree has at most $n - 1$ internal nodes. If each obligation has at most one final submission per epoch and waves have width w , then $K_e \leq \lceil n/w \rceil$ for the evaluated sequential grouping rule. Hence

$$0 \leq L_e(T) \leq B = (1 + \lceil n/w \rceil)(n - 1)C. \quad (11)$$

This is a valid but loose range for the sufficient promotion gate. A narrower empirically estimated range is not automatically a valid replacement for a proved bound. The packet cap limits local fanout, whereas increasing w reduces update frequency and increases the number of completions that may remain unreported before a flush. These are different controls. A tree comparison is meaningful only at the same batching policy or with an explicit freshness/latency trade-off.

A.3 Exact recurrence and reconstruction

The forest recurrence stores a split location for every finite $F_k(i, j)$ and the selected root arity for every $D(i, j)$. Reconstruction first partitions an interval into its selected k child intervals and then recursively reconstructs each subtree. Any tie can be resolved deterministically; the reported empirical cost remains optimal. The artifact uses a fixed numerical tie rule and stores every fitted tree as an explicit JSON structure. An independent audit walks that structure, verifies the partition, and recomputes traffic directly from recorded waves.

The counterexample in Theorem 3 is checked by exhaustive enumeration of all five ordered binary trees on four leaves. More generally, the tests compare the frontier compiler with exhaustive enumeration on two through seven leaves. These executable checks do not replace the proof, but they detect recurrence, indexing, normalization, and reconstruction mistakes.

A.4 Safety under reordered and delayed messages

A task result carries its obligation index, epoch token, and exact input binding. If an old result arrives after a new epoch begins, the epoch check rejects it before it changes current evidence. An ABA transition still increments the epoch; matching source bytes alone cannot resurrect the earlier ticket. If the same final record is delivered twice, the second delivery is idempotent. If two different records claim the same obligation in one epoch, the guard revokes authorization, retaining failure when either record fails and otherwise marking the obligation unknown. Further attempts require a new epoch.

For an external integration, this local rule is necessary but insufficient. The merge or deployment service must compare the candidate identity and integration-head version atomically with its write. The supplied CLI emits an authorization report but deliberately performs no external merge. A user cannot infer distributed linearizability from a passing local race test.

Table 7: Scope families; exact path arrays and node IDs are stored in the artifact.

Subject	Mutated source families	Approved test families
NetworkX	Connected, strongly/weakly connected components; breadth/depth traversal; unweighted shortest paths	Component and traversal suites; unweighted and generic shortest-path tests
Toolz	Iterable, functional, dictionary, recipes, utilities	Iterable, functional, dictionary, recipes, curried, utility suites
Future Code	Contracts, context, coordination, security, reporting, store	Contracts, store, contract-first execution, coordination, coordination edges, hierarchy contracts

B Exact Experimental Scope and Reproduction

B.1 Source and test scopes

The source snapshots are bundled rather than fetched at reproduction time. Package versions and the actual environment are recorded; installation of external dependencies remains subject to availability in the reproducer’s environment. This study used Python 3.13.5 on Linux. No claim is made that Windows or an accelerator runtime was tested. The code does not require a GPU or NPU.

The first experimental plan, the chosen mutation spans, the later confirmation freeze, all candidate hashes, raw pytest records, fitted plans, direct timing records, and the independent audit output are included. Validation logic rejects unknown registry IDs, duplicate interventions, nonfinite cost inputs, and order lists that omit tests. The sole unresolved development candidate is retained in the source plan and oracle logs.

B.2 Reproduction tiers

The least expensive check runs the kernel/compiler/wrapper tests and verifies the release manifest. The next tier reruns the independent stored-evidence audit without executing project code. A complete experimental reproduction executes source mutations and direct timing in fresh copies, then regenerates tables. Source preparation and test execution are recorded separately; stored paper measurements are never overwritten implicitly. The README provides exact commands and the archive verifier checks packaged-file identities.

```
python -m pytest -q tests tests_v1
python audit/independent_check.py
python -m experiments.summarize
```

The full experimental scripts refuse to overwrite certain frozen outputs. The supplied preparation helper creates a new research directory and results directory, retaining the original baseline profiles and fitted plans for fixed-policy re-execution. Its safe preparation path is tested; the entire campaign was not run a second time through that convenience launcher during packaging. Expensive or destructive external actions are not part of reproduction. A practical local use of the wrapper is:

```
python -m ichact init --project /work/trusted-baseline \
  --contract /work/contracts/approved.json \
  --output /work/reports/baseline --test-path tests

python -m ichact verify --project /work/candidate \
  --contract /work/contracts/approved.json \
  --output /work/reports/candidate-001
```

An order file, when supplied, must be an exact JSON permutation of the approved registry. The report explicitly distinguishes a pass, a failed candidate, and an unresolved execution. For untrusted source code,

Table 8: Exploratory batching sensitivity on the same 33 confirmation cases. Each row refits on training waves only. Values are computed from the exact packet-price formula; batching changes progress granularity and is not a free speedup.

Completions/wave	Cases	Fixed KiB	Fitted KiB	Reduction (%)
1	33	466.4	431.2	7.55
8	33	234.5	210.3	10.33
32	33	149.2	135.5	9.19
End only	33	96.7	93.6	3.19

Table 9: Functional wrapper smoke with actual project tests. One known rejected confirmation mutant per project is selected for demonstration, then the original source is restored. These are single runs, not a sampled timing benchmark or autonomous repairs.

Subject	Stage	Tests	Verdict	Process s	Wrapper s
NetworkX	Baseline	170	PASS	2.777	3.265
NetworkX	Known fault	1	FAIL	2.574	3.068
NetworkX	Restoration	170	PASS	3.033	3.550
Toolz	Baseline	148	PASS	1.473	1.784
Toolz	Known fault	43	FAIL	1.471	1.771
Toolz	Restoration	148	PASS	1.423	1.743
Future Code	Baseline	138	PASS	2.824	3.160
Future Code	Known fault	1	FAIL	1.321	1.665
Future Code	Restoration	138	PASS	2.675	2.996

run this wrapper within an appropriate container or VM and keep credentials, approval files, and the checker outside candidate control. The wrapper is intentionally conservative about writes to its snapshot: even a generated source-tree file can prevent a pass until the operator establishes an approved output policy.

C Additional Diagnostics and Sensitivity

The final development-tree validation suite passes all 144 tests. It includes the original 90 HACT tests plus 54 tests for frontiers, ordering, epochs, conflicting results, identities and the strict wrapper. A separate original-runtime run passes 263 tests. The latter is not added to the 456 scoped baseline tests as though they were all distinct. The independent stored-evidence audit verifies 121 nonoverlapping source interventions, 300 certificate decisions, 384 actual timing-run verdicts, and 17 paired statistical comparisons, including recomputation of the confidence intervals. A deliberately changed packet total is rejected by that independent path.

The original capture attempt for the full Future Code suite produced an incomplete result and is retained as a diagnostic, not a successful baseline. A later complete run with file-backed subprocess output passed all 263 tests. The mutation campaign uses the explicitly documented 138-test core scope. Incomplete executions are never filled with fabricated outcomes.

The generic wrapper also rejects modified protected acceptance inputs on all three demonstrated projects. The unit tests cover a malformed order that tries to omit a test and a candidate that writes into its checked source snapshot; neither receives a pass. These are tested local boundaries, not proof against a hostile process with access to the checker or the host.

D Artifact and Claim Ledger

Table 10: What each evidence source establishes, and what it does not.

Evidence	Supported claim	Unsupported extrapolation
Mathematical proofs	Exact objective/DP and conditional predicate preservation	Complete program correctness or complete causal discovery
Source mutation oracles	Outcomes on identified source copies and scoped existing tests	Representative real-world bug/repair distribution
Direct timing	Observed process costs for the frozen cohorts and orderings	Total development cost, API billing, or universal speedup
Packet replay	Exact bytes for the stated kernel format and completion waves	Measured wide-area bandwidth or LLM token savings
Protocol stress tests	Rejection of tested stale, missing, conflicting and invalid records	Adversarial remote attestation or distributed consensus
Wrapper smoke	Local baseline–fault–restoration lifecycle with source checks	Autonomous repair or production deployment integration

The artifact contains a SHA-256 file manifest and a separate release verification record. The final test suite is rerun from the extracted deliverable. Generated tables are driven by recorded measurements, and the independent audit does not import the producer’s optimizer or cost helpers. These controls improve traceability but are not a substitute for an independent external replication or a security assessment.